Relatório – Projeto II

Gabriel Carraro Salzedas – 16827905

Arthur de Castro Dias – 16855302

Descrição do problema

Ordenar, por sua definição, é organizar uma sequência de elementos de modo que eles estabeleçam uma relação de ordem.

Ordenação é um tema muito discutido dentro do ramo da computação, uma vez que sempre houve uma grande procura em busca de algoritmos mais eficientes e eficazes. Com o passar do tempo, diversos algoritmos foram criados e modificados em busca de uma melhor performance para o problema.

Um dos motivos para a importância de ordenar vetores é a facilidade que vetores ordenados geram para outras funções, como a busca. Uma característica interessante dos métodos de ordenação é a sua independência da complexidade dos registros desordenados.

Apesar do problema ser único, os métodos de resolução são diversos, desde simplesmente trocar elementos consecutivos desordenados até a verificar dígitos na mesma posição de diferentes números. Todos são métodos válidos que trazem consigo distintos tempos de execução e memória utilizada.

Com isso, este trabalho foi desenvolvido em busca de analisar os diferentes métodos de ordenação e suas características básicas, além de suas performances em melhores, piores e médios casos.

Além disso, para o cálculo de tempo de execução, fizemos uso do seguinte código, aplicado para cada *sort*:

```
clock_t inicio = clock();

mergesort(v2, n);

clock_t fim = clock();
double tempo_execucao = (double)(fim - inicio) / CLOCKS_PER_SEC;
```

1. Bubblesort

```
void bubblesort(int *v, unsigned int n) {
    bool trocou = 1;
    for(unsigned int i = 0; i<n-1 && trocou; ++i) {
        trocou = 0;
        for(unsigned int j = 0; j<n-i-1; ++j) {
            ++nComparacoes;
            if(v[j]>v[j+1]) {
                int temp = v[j];
                v[j] = v[j+1];
                v[j+1] = temp;
                nMovimentacoes += 3;
                trocou = 1;
            }
        }
    }
}
```

O primeiro método de ordenação feito foi o *BubbleSort* aprimorado: é um método estável e simples que se baseia na verificação do elemento atual com seu sucessor e, caso aquele seja maior, é realizada a troca de posição entre os dois; o *BubbleSort* garante que, a cada passada, o maior valor é armazenado ao final do subvetor verificado.

O exemplo dado é o aprimorado, pois ele apresenta maior desempenho que sua forma simples, já que este não possui a *flag "trocou"* para verificar se houve trocas de posições durante a execução; dessa maneira, várias verificações desnecessárias podem ser evitadas.

Cada vez que comparamos duas chaves, incrementamos *nComparacoes* e, após cada troca, incrementamos *nMovimentacoes* em 3, uma vez que cada troca envolve 3 movimentações. Noto aqui que *nComparacoes* e *nMovimentacoes* são variáveis globais.

Vamos analisar para os vetores ordenados:

Tamanhos	nComparações	nMovimentações	Tempo de execução (s)
100	99	0	0
1000	999	0	0
10000	9.999	0	0
100000	99.999	0	0

Como sabemos pelo funcionamento do *Bubblesort* aprimorado, se o vetor já está ordenado, ele executará em $\Omega(n)$: simplesmente irá passar por todos os elementos e não fazer nenhuma movimentação. O tempo de execução aqui foi tão baixo que o programa o retornou como 0 absoluto.

Agora, vamos ver para os vetores inversos:

Tamanhos	nComparações	nMovimentações	Tempo de execução (s)
100	4.922	14.670	0
1000	449.472	1.493.820	0,001
10000	49.994.972	149.984.820	0,103
100000	4.999.949.972	14.999.849.820	14,34

Novamente, como sabemos do *Bubblesort*, o pior caso é $O(n^2)$ – caso esse que ocorre justamente quando o vetor está inversamente ordenado. Na prática, a quantidade comparações que ocorrem no vetor completamente inverso é $\frac{n\cdot(n-1)}{2}$ e a quantidade de movimentações seria o triplo disso. Duas coisas podem ser notadas, no entanto: o número de comparações não é exatamente o dado pela equação e nem mesmo a quantidade movimentações é o triplo quer seja da quantidade de comparações ideal ou da realmente obtida.

Em teoria, deveríamos obter 4.950 comparações e 14.850 movimentações. A razão pela qual isso não ocorreu é porque, apesar de nosso vetor estar inversamente ordenado, há repetições de alguns valores. Por exemplo, o número "0" aparece 9 vezes. Isso ocorreu pela maneira que decidimos gerar os arquivos com os vetores e poderíamos facilmente ter gerado de modo a garanti que cada elemento fosse único, mas julgamos essa alternativa como sendo mais interessante (comentaremos um pouco sobre como geramos os vetores ao final do documento).

Apenas para mostrar como isso de fato aconteceria, removemos todos os elementos duplicados do vetor de tamanho 100, chegando a um vetor de tamanho 76. O código retornou 2.850 comparações e 8.550 movimentações: $38\cdot75 = 2.850$ e $2.850\cdot3 = 8.550$; assim sendo, verificamos que a conta de fato está correta.

Finalmente, analisemos os casos de vetores com elementos aleatórios:

Tamanhos	nComparações	nMovimentações	Tempo de execução (s)
100 (a)	4.922	7.584	0
100 (b)	4.779	7.173	0
100 (c)	4.859	7.425	0
100 (d)	4.949	7.170	0
100 (e)	4.872	7.776	0
1000 (a)	497.609	742.467	0,001
1000 (b)	498.905	738.036	0,001
1000 (c)	499.422	760.695	0,001
1000 (d)	499.329	734.664	0,001
1000 (e)	499.175	762.687	0,001
10000 (a)	49.973.472	75.067.398	0,102
10000 (b)	49.979.424	73.936.272	0,099
10000 (c)	49.976.855	74.940.867	0,103
10000 (d)	49.993.725	75.177.969	0,102
10000 (e)	49.976.855	74.885.406	0,102

100000 (a)	4.999.877.229	7.519.165.392	22,004
100000 (b)	4.998.945.804	7.502.318.994	21,855
100000 (c)	4.999.635.179	7.543.294.377	23,686
100000 (d)	4.999.845.804	7.501.218.309	18,122
100000 (e)	4.999.843.970	7.502.085.201	21,925

Primeiramente, vale notar como o caso aleatório – ou caso médio – se aproxima muito com o pior caso em todas as situações quanto à quantidade de comparações e quanto ao tempo de execução. A quantidade de movimentações, entretanto, é aproximadamente 50% da realizada para o vetor inverso.

Algo interessante de se visualizar é que para o n = 100.000, o tempo de execução dos vetores aleatórios foi consideravelmente maior do que o vetor inverso, mesmo realizando menos movimetações e comparações. Isso ocorre porque as trocas estão mais espalhadas, gerando mais *cache misses*, fazendo o processador gastar mais tempo com esses acessos irregulares; além disso, o processador consegue prever muito bem o próximo movimento no caso inverso, o que não é verdade para o caso aleatório, em que ele erra muitas previsões, gastando ainda mais tempo.

2. Selectionsort

```
void selectionsort(int *v, unsigned int n) {
    for(unsigned int i = 0; i<n-1; ++i) {
        unsigned int min = i;
        for(unsigned int j = i+1; j<n; ++j) {
            ++nComparacoes;
            if(v[j]<v[min]) {
                min = j;
            }
        }
        if(i!=min) {
            int temp = v[i];
            v[i] = v[min];
            v[min] = temp;
            nMovimentacoes += 3;
        }
    }
}
```

O *selectionsort* é um método instável que procura o menor elemento do vetor e o posiciona na primeira posição do vetor. A cada iteração, o tamanho do subvetor analisado vai diminuindo, garantindo que o vetor termine sempre ordenado ao final da execução.

Implementamos aqui um *selectionsort* convencional, incrementando a quantidade de comparações a cada comparação entre v[j] e v[min] e incrementando em 3 a quantidade de movimentações (cada troca faz 3 movimentações).

Vamos analisar para os vetores ordenados:

Tamanhos	nComparações	nMovimentações	Tempo de execução (s)
100	4.950	0	0
1000	499.500	0	0
10000	49.995.000	0	0,073
100000	4.999.950.000	0	7,96

Vamos analisar para os vetores inversos:

Tamanhos	nComparações	nMovimentações	Tempo de execução (s)
100	4.950	198	0
1000	499.500	1.548	0
10000	49.995.000	15.048	0,074
100000	4.999.950.000	150.048	7,824

Note que, no *Selectionsort*, o número de comparações, no pior e melhor casos é $\frac{n\cdot(n-1)}{2}$, tornando o *Selectionsort* $\Theta(n^2)$. Entretanto, ele faz menos movimentações que o *Bubblesort* (3 vezes a quantidade de comparações), fazendo apenas 3(n-1) movimentações no pior caso e, assim, garantindo um tempo de execução menor que o do *bubble* no pior caso.

Agora, os casos de vetores com elementos aleatórios:

Tamanhos	**nComparações**	**nMovimentações**	**Tempo de execução (s)**
100 (a)	4.950	294	0
100 (b)	4.950	291	0
100 (c)	4.950	291	0
100 (d)	4.950	285	0
100 (e)	4.950	282	0
1000 (a)	499.500	2.985	0,001
1000 (b)	499.500	2.982	0,001
1000 (c)	499.500	2.973	0,001
1000 (d)	499.500	2.979	0,001
1000 (e)	499.500	2.976	0,001
10000 (a)	49.995.000	29.967	0,073
10000 (b)	49.995.000	29.961	0,074
10000 (c)	49.995.000	29.967	0,074
10000 (d)	49.995.000	29.979	0,073
10000 (e)	49.995.000	29.964	0,074
100000 (a)	4.999.950.000	299.943	9,065
100000 (b)	4.998.950.000	299.946	8,659
100000 (c)	4.999.955.000	299.952	11,392
100000 (d)	4.999.950.000	299.970	9,259
100000 (e)	4.999.950.000	299.955	7,895

A quantidade de movimentações aqui foi um pouco maior do que para os casos análogos do vetor inverso, com tempo de execução semelhante ou maior.

3. Insertionsort

```
void insertionsort(int *v, unsigned int n) {
    for(unsigned int i = 1; i<n; ++i) {
        int temp = v[i];
        unsigned int j;
        for(j = i; j>0 && temp<v[j-1]; --j) {
            ++nComparacoes;
            v[j] = v[j-1];
            ++nMovimentacoes;
        }
        v[j] = temp;
        ++nMovimentacoes;
    }
}
```

O *insertionsort* é um método estável que se baseia na técnica de percorrer subvetores até que seja encontrado o primeiro elemento fora de ordem; após isso, ele é posicionado na sua localização correta.

Aqui, incrementamos a quantidade de comparações a cada comparação de temp com v[j-1] e a quantidade de movimentações após cada trocar de v[j] com v[j-1] e de v[j] com temp.

Vamos analisar para os vetores ordenados:

Tamanhos	nComparações	nMovimentações	Tempo de execução (s)
100	99	99	0
1000	999	999	0
10000	9999	9999	0
100000	99999	99999	0

Para os vetores inversos:

Tamanhos	nComparações	nMovimentações	Tempo de execução (s)
100	4914	4989	0
1000	499.464	500.439	0,001
10000	49.994.964	50.004.939	0,079
100000	4.999.949.964	5.000.049.939	10,738

Para os vetores aleatórios:

Tamanhos	nComparações	nMovimentações	Tempo de execução (s)
100 (a)	2.625	2.627	0
100 (b)	2.485	2.490	0
100 (c)	2.570	2.574	0

100 (d)	2.487	2.489	0
100 (e)	2.687	2.691	0
1000 (a)	248.482	248.488	0,001
1000 (b)	246.999	247.011	0
1000 (c)	254.556	254.564	0
1000 (d)	245.882	245.887	0
1000 (e)	255.220	255.288	0
10000 (a)	25.032.455	25.032.465	0,041
10000 (b)	24.655.412	24.655.423	0,041
10000 (c)	24.990.276	24.990.288	0,04
10000 (d)	25.069.317	25.069.322	0,042
10000 (e)	24.971.793	24.971.801	0,039
100000 (a)	2.506.488.459	2.506.488.463	5,53
100000 (b)	2.500.872.988	2.500.872.997	4,271
100000 (c)	2.514.531.449	2.514.531.458	5,067
100000 (d)	2.500.506.095	2.500.506.102	5,791
100000 (e)	2.500.795.052	2.500.795.066	5,434

É interessante notar como a quantidade de comparações e de movimentações no *Insertionsort* são quase idênticas em quaisquer situações, uma vez que cada comparação bem-sucedida causa uma movimentação e cada iteração gera ao menos uma comparação e uma movimentação.

No vetor já ordenado, cada elemento só precisa de 1 comparação, resultando em *n-1* comparações; e, mesmo sem deslocar nada, o algoritmo sempre faz a atribuição de *v[j]* com *temp*, levando a, também, *n-1* movimentações.

Já no caso do vetor inverso, cada elemento precisa ser comparado com todos os anteriores, levando-nos a $\frac{n\cdot(n-1)}{2}$comparações. Como cada comparação verdadeira leva a um deslocamento, somando isso à movimentação que sempre ocorre no final, teremos $\frac{n\cdot(n-1)}{2} + (n-1)$ deslocamentos. Como temos elementos duplicados, essa conta não baterá perfeitamente com os resultados obtidos.

No caso do vetor aleatório, os dois valores também andam juntos porque cada comparação que dá verdadeiro gera um deslocamento, e cada iteração sempre termina com uma inserção. Vale notar que o número de comparações e deslocamentos do vetor aleatório foi aproximadamente metade do vetor inverso (pior caso).

4. Shellsort

```
void shellsort(int *v, unsigned int n) {
    for(unsigned int h = n/2; h>0; h /= 2) {
        for(unsigned int i = h; i<n; ++i) {
            int temp = v[i];
            unsigned int j = i;
            while (j >= h) {
                ++nComparacoes;
                if (v[j-h] > temp) {
                    v[j] = v[j-h];
                    ++nMovimentacoes;
                    j -= h;
                } else {
                    break;
                }
            }
            v[j] = temp;
            ++nMovimentacoes;
        }
    }
}
```

O *shellsort* é um método instável que generaliza o *insertionsort*, mas trabalha com intervalos maiores para comparação dos elementos, de modo que tais intervalos vão diminuindo com o tempo, utilizando diferentes decrementos, cada vez menores, até que todos os itens sejam comparados entre si.

Vamos analisar para os vetores ordenados:

Tamanhos	nComparações	nMovimentações	Tempo de execução (s)
100	503	503	0
1000	8.006	8.006	0
10000	120.005	120.005	0
100000	1.500.009	1.500.006	0,011

Para os vetores inversos:

Tamanhos	nComparações	nMovimentações	Tempo de execução (s)
100	660	751	0
1000	11.698	12.686	0
10000	172.656	182.551	0
100000	2.244.561	2.344.541	0,004

Para os vetores aleatórios:

Tamanhos	nComparações	nMovimentações	Tempo de execução (s)
100 (a)	872	920	0
100 (b)	832	879	0
100 (c)	863	919	0
100 (d)	791	830	0
100 (e)	921	979	0
1000 (a)	15.122	15.635	0
1000 (b)	14.706	15.220	0
1000 (c)	14.736	15.256	0
1000 (d)	15.127	15.634	0
1000 (e)	15.328	15.856	0
10000 (a)	264.556	269.591	0,001
10000 (b)	257.264	262.290	0,001
10000 (c)	265.542	270.644	0,001
10000 (d)	262.487	267.635	0,001
10000 (e)	281.673	286.725	0,001
100000 (a)	4.104.435	4.155.090	0,02
100000 (b)	4.611.748	4.661.971	0,025
100000 (c)	4.262.311	4.313.131	0,027
100000 (d)	4.291.102	4.341.466	0,015
100000 (e)	4.314.828	4.365.320	0,017

Com o *gap* que escolhemos para esse *Shellsort* – a original divisão por 2, o pior caso é O(n^2). Para n = 10.000, isso seria 100.000.000; note, contudo, que chegamos em quantidades bem menores, uma vez que esse valor ocorre em casos muito específicos. Além disso, esse é um dos piores *gaps* conhecidos para o *Shellsort*, mas optamos por ele por ser o original. Ainda assim, obtivémos resultados bem melhores que os anteriores.

Vale notar que a quantidade de comparações e movimentações é muito similar, já que para cada *while* há uma comparação e, dentro, uma movimentação se a comparação for verdadeira mais uma movimentação depois da execução do *while*.

5. Quicksort

```
void quicksort(int *v, unsigned int esq, unsigned int dir) {
    if (esq >= dir) {
        return;
    }

    unsigned int i = esq, j = dir;
    unsigned int meio = (esq + dir) / 2;

    if (v[esq] > v[meio]) {
        int temp = v[esq];
        v[esq] = v[meio];
        v[meio] = temp;
        nMovimentacoes += 3;
    }

    if (v[esq] > v[dir]) {
        int temp = v[esq];
        v[esq] = v[dir];
        v[dir] = temp;
        nMovimentacoes += 3;
    }

    if (v[meio] > v[dir]) {
        int temp = v[meio];
        v[meio] = v[dir];
        v[dir] = temp;
        nMovimentacoes += 3;
    }

    nComparacoes += 3;

    int pivo = v[meio];

    do {
        while (v[i] < pivo) {
            ++nComparacoes;
            ++i;
        }
        ++nComparacoes;

        while (v[j] > pivo) {
            ++nComparacoes;
            --j;
        }
        ++nComparacoes;
```

```
        ++nComparacoes;
        if (i <= j) {
            if (i < j) {
                int temp = v[i];
                v[i] = v[j];
                v[j] = temp;
                nMovimentacoes += 3;
            }
            ++i;
            if (j == 0) {
                break;
            }
            --j;
        }
    } while (i < j);

    if (j > esq) {
        quicksort(v, esq, j);
    }
    if (i < dir) {
        quicksort(v, i, dir);
    }
}
```

No *quicksort,* temos um método instável que faz uso da escolha de um pivô, o qual serve de elemento modelo, sendo que quaisquer elementos maiores que ele devem ser posicionados após ele e os menores devem ser colocados antes dele. Dessa maneira, ao dividir o vetor em subvetores cada vez menores, terminamos por ter o vetor ordenado.

O uso do cálculo da mediana impede a ocorrência do pior caso (aquele em que o elemento escolhido é o maior ou menor item do vetor); nesse caso, um dos *loops* acabaria por apenas terminar quando encontrasse o pivô. Já que não haveria elementos maiores ou menores a depender do caso, isso causaria uma complexidade $O(n^2)$.

Para a mediana de três, fazemos 3 comparações e, em cada uma, fazemos uma troca (3 movimentações) se a comparação for verdadeira. Para cada comparação de v[i] com o pivô, incrementamos a variável e fazemos uma troca (3 movimentações) quando encontramos um elemento maior que o pivô à esquerda e um menor que o pivô à direita.

Vamos analisar para os vetores ordenados:

Tamanhos	**nComparações**	**nMovimentações**	**Tempo de execução (s)**
100	1.390	612	0
1000	12.774	1.212	0
10000	150.555	1.212	0

100000	1.863.442	1.212	0,003

É interessante avaliar de onde vieram essas movimentações e por que elas acabam ficando constantes. Como já dissemos, temos alguns elementos iguais (que, pela natureza da geração do vetor, ficam concentrados nos primeiros elementos). Isso faz com que esse *quicksort*, que não é estável, faça trocas quando dois elementos são iguais, causando movimentações.

Para os vetores inversos:

Tamanhos	**nComparações**	**nMovimentações**	**Tempo de execução (s)**
100	1.067	282	0
1000	11.750	1.632	0
10000	154.421	15.132	0,001
100000	1.912.314	150.132	0,003

Para os vetores aleatórios:

Tamanhos	**nComparações**	**nMovimentações**	**Tempo de execução (s)**
100 (a)	1.147	519	0
100 (b)	1.220	558	0
100 (c)	1.223	591	0
100 (d)	1.168	510	0
100 (e)	1.214	540	0
1000 (a)	16.642	8.004	0,001
1000 (b)	16.294	8.088	0
1000 (c)	17.464	7.953	0
1000 (d)	17.759	7.707	0
1000 (e)	16.543	7.980	0,001
10000 (a)	216.664	104.316	0,001
10000 (b)	224.242	103.242	0,001
10000 (c)	212.764	105.726	0,001
10000 (d)	217.515	104.763	0,001
10000 (e)	216.298	104.490	0,001
100000 (a)	2.881.106	1.390.755	0,013
100000 (b)	2.809.098	1.400.322	0,009
100000 (c)	2.803.757	1.397.661	0,011
100000 (d)	2.833.091	1.394.238	0,008
100000 (e)	2.790.360	1.404.177	0,008

Uma coisa interessante de se notar é que os vetores inversos e ordenados ficam muito próximos em quantidade de comparações, enquanto os vetores aleatoriamente ordenados têm muito mais comparações e movimentações.

Isso ocorre porque escolhemos o pivô como sendo a mediana de três, o que funciona maravilhosamente bem para os vetores em ordem crescente e decrescente, uma vez que encontra, de fato, a mediana; contudo, isso não é verdade para o vetor aleatório. Ainda assim, a mediana de três ainda funciona muito bem mesmo para o vetor aleatório, resultando em ordenações muito rápidas.

6. Heapsort

```
void rearranjar_heap(int *v, unsigned int i, unsigned int n) {
    unsigned int esq = 2*i + 1, dir = esq + 1, maior = i;
    int temp;

    if (esq < n && v[esq] > v[maior]) {
        maior = esq;
    }

    if (dir < n && v[dir] > v[maior]) {
        maior = dir;
    }

    if (maior != i) {
        temp = v[i];
        v[i] = v[maior];
        v[maior] = temp;
        nMovimentacoes += 3;
        rearranjar_heap(v, maior, n);
    }

    nComparacoes += 3;
}

void heapsort(int *v, unsigned int n) {
    // Construção do max-heap (sem underflow)
    for (unsigned int i = n/2; i-- > 0;) {
        rearranjar_heap(v, i, n);
    }

    // Extração
    for (unsigned int fim = n - 1; fim > 0; --fim) {
        int temp = v[0];
        v[0] = v[fim];
        v[fim] = temp;
        nMovimentacoes += 3;
        rearranjar_heap(v, 0, fim);
    }
}
```

O *heapsort* é um método instável que se baseia na montagem de uma árvore *heap*. No nosso caso, uma *heap_max*, na qual o maior elemento fica na raiz e os pais são sempre maiores que seus filhos; dessa maneira, podemos pegar o primeiro elemento do vetor e posicioná-lo no fim

para que, assim, os maiores elementos sejam sempre armazenados no fim de cada subvetor utilizado e a função *rearranjar_heap* garanta que a estrutura será sempre seguida para que o algoritmo não gere uma ordenação errada.

Em *rearranjar_heap*, fazemos 3 comparações e uma troca (3 movimentações) quando temos que mudar o maior elemento. Já em *heapsort*, fazemos uma troca para cada iteração do *for*.

Analisemos os vetores ordenados:

Tamanhos	**nComparações**	**nMovimentações**	**Tempo de execução (s)**
100	1.608	1.458	0
1000	29.988	28.488	0
10000	410.292	395.292	0,001
100000	5.102.082	4.952.082	0,011

Para os vetores inversos:

Tamanhos	**nComparações**	**nMovimentações**	**Tempo de execução (s)**
100	1.620	1.470	0
1000	26.388	24.888	0
10000	365.016	350.016	0,001
100000	4.642.239	4.492.239	0,01

Para os vetores aleatórios:

Tamanhos	**nComparações**	**nMovimentações**	**Tempo de execução (s)**
100 (a)	1.923	1.773	0
100 (b)	1.917	1.767	0
100 (c)	1.911	1.761	0
100 (d)	1.896	1.746	0
100 (e)	1.836	1.686	0
1000 (a)	28.764	27.264	0
1000 (b)	28.839	27.339	0
1000 (c)	28.782	27.282	0
1000 (d)	28.749	27.249	0,001
1000 (e)	28.755	27.255	0,001
10000 (a)	387.846	372.846	0,001
10000 (b)	387.984	372.984	0,001
10000 (c)	387.612	372.612	0,001
10000 (d)	387.741	372.741	0,001
10000 (e)	387.795	372.795	0,001
100000 (a)	4.874.112	4.724.112	0,013
100000 (b)	4.875.282	4.725.282	0,024
100000 (c)	4.873.779	4.723.779	0,016
100000 (d)	4.873.920	4.723.920	0,019

100000 (e)	4.874.034	4.724.032	0,022

Como a complexidade do heapsort é $n \cdot \log_2 n$ em todos os casos, a quantidade de comparações e movimentações são muito similares em qualquer uma das três situações.

7. Mergesort

```
void mergesort(int *v, unsigned int n) {
    int *temp = malloc(n * sizeof(int));
    if (temp == NULL) {
        return;
    }

    int *origem = v;
    int *destino = temp;

    for (unsigned long long i = 2; (i / 2) < n; i *= 2) {

        for (unsigned long long j = 0; j < n; j += i) {
            unsigned long long meio = j + (i / 2);
            unsigned long long final = j + i;

            if (meio > n){
                meio = n;
            }
            if (final > n) {
                final = n;
            }

            unsigned long long k = j, l = meio, m = j;

            while (k < meio && l < final) {
                ++nComparacoes;
                if (origem[k] <= origem[l]) {
                    destino[m++] = origem[k++];
                } else {
                    destino[m++] = origem[l++];
                }
                ++nMovimentacoes;
            }

            while (k < meio) {
                ++nComparacoes;
                destino[m++] = origem[k++];
                ++nMovimentacoes;
            }
            while (l < final) {
                ++nComparacoes;
                destino[m++] = origem[l++];
                ++nMovimentacoes;
            }
            nComparacoes += 2;
        }
```

```
        int *temp2 = origem;
        origem = destino;
        destino = temp2;
    }

    if (origem != v) {
        for (unsigned int i = 0; i < n; ++i) {
            v[i] = origem[i];
            ++nMovimentacoes;
        }
    }

    free(temp);
}
```

O *mergesort* é um método estável que se baseia na ideia de dividir o vetor no meio e, durante a remontagem do vetor original, comparar cada subvetor entre si, fazendo ordenações em menor escala, as quais garantem que o vetor estará sempre ordenado ao final.

Casos com vetores ordenados:

Tamanhos	nComparações	nMovimentações	Tempo de execução (s)
100	904	800	0
1000	12.002	10.000	0
10000	16.010	14.000	0
100000	1.000.012	1.800.000	0,003

Casos com vetores inversos:

Tamanhos	nComparações	nMovimentações	Tempo de execução (s)
100	904	800	0
1000	12.002	10.000	0
10000	160.010	140.000	0
100000	1.900.012	1.800.000	0,003

Casos com vetores aleatórios:

Tamanhos	nComparações	nMovimentações	Tempo de execução (s)
100 (a)	904	800	0
100 (b)	904	800	0
100 (c)	904	800	0
100 (d)	904	800	0
100 (e)	904	800	0
1000 (a)	12.002	10.000	0
1000 (b)	12.002	10.000	0

1000 (c)	12.002	10.000	0
1000 (d)	12.002	10.000	0
1000 (e)	12.002	10.000	0
10000 (a)	160.010	140.000	0,001
10000 (b)	160.010	140.000	0,001
10000 (c)	160.010	140.000	0,001
10000 (d)	160.010	140.000	0,001
10000 (e)	160.010	140.000	0,001
100000 (a)	1.900.012	1.800.000	0,009
100000 (b)	1.900.012	1.800.000	0,009
100000 (c)	1.900.012	1.800.000	0,009
100000 (d)	1.900.012	1.800.000	0,009
100000 (e)	1.900.012	1.800.000	0,009

Note que esse *mergesort* executa sempre em O($n \cdot \log_2 n$), independentemente do vetor original, levando-nos a quantidades de comparação e movimentação iguais para um mesmo n.

8. Contagem de menores

```
void contagem_menores(int *v, unsigned int n) {
    int *X = calloc(n,sizeof(int));
    if(X == NULL) {
        return;
    }
    int *B = malloc(n*sizeof(int));
    if(B == NULL) {
        free(X);
        return;
    }

    for(unsigned int i=1; i<n; ++i) {
        for(unsigned int j=i; j>0; --j) {
            ++nComparacoes;
            if(v[i]<v[j-1]) {
                X[j-1]++;
                ++nMovimentacoes;
            } else {
                X[i]++;
                ++nMovimentacoes;
            }
        }
    }

    for(int i=0; i<n; ++i) {
        B[X[i]] = v[i];
        ++nMovimentacoes;
    }

    for(int i=0; i<n; ++i) {
        v[i] = B[i];
        ++nMovimentacoes;
    }
    free(X);
    free(B);
}
```

A contagem de menores é um método estável de ordenação que tem por ideia principal a contabilização de quantos itens de um vetor são menores que o elemento analisado. Ao final, ele retorna o índice que os elementos deveriam ser posicionados, então só devemos percorrer o vetor e colocá-los em suas devidas posições de acordo com a contagem. O maior problema desse método é seu uso excessivo da memória.

Vetores ordenados:

Tamanhos	nComparações	nMovimentações	Tempo de execução (s)
100	4.950	5.150	0
1000	499.500	501.500	0
10000	49.995.000	50.015.000	0,082
100000	4.999.950.000	5.000.150.000	9,057

Vetores inversos:

Tamanhos	nComparações	nMovimentações	Tempo de execução (s)
100	4.950	5.150	0
1000	499.500	501.500	0
10000	49.995.000	50.015.000	0,078
100000	4.999.950.000	5.000.150.000	9,648

Vetores aleatórios:

Tamanhos	nComparações	nMovimentações	Tempo de execução (s)
100 (a)	4.950	5.150	0
100 (b)	4.950	5.150	0
100 (c)	4.950	5.150	0
100 (d)	4.950	5.150	0
100 (e)	4.950	5.150	0
1000 (a)	499.500	501.500	0
1000 (b)	499.500	501.500	0
1000 (c)	499.500	501.500	0
1000 (d)	499.500	501.500	0
1000 (e)	499.500	501.500	0
10000 (a)	49.995.000	50.015.000	0,158
10000 (b)	49.995.000	50.015.000	0,153
10000 (c)	49.995.000	50.015.000	0,155
10000 (d)	49.995.000	50.015.000	0,155
10000 (e)	49.995.000	50.015.000	0,157
100000 (a)	4.999.950.000	5.000.150.000	16,668
100000 (b)	4.999.950.000	5.000.150.000	20,822
100000 (c)	4.999.950.000	5.000.150.000	23,073
100000 (d)	4.999.950.000	5.000.150.000	17,062
100000 (e)	4.999.950.000	5.000.150.000	19,591

Note que esse algoritmo também depende somente de n, de modo que a ordem original dos elementos é indiferente. O laço externo executa $n-1$ vezes, o interno i vezes para cada i, nos dando:

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

Dentro do laço interno, há sempre uma comparação (que ocorre, portanto, $\frac{n(n-1)}{2}\ vezes$).

Independentemente do resultado da comparação, há sempre um incremento (um se menor e outro se maior ou igual) – ou seja, cada comparação acarreta em exatamente uma modificação. Além disso, temos as comparações fora desse laço externo que também dependem somente de n.

Dessa maneira, nosso algoritmo é O($\frac{n(n-1)}{2}$) $= O(n^2)$. Como a ordenação independe do estado do vetor original, o número de comparações e modificações é fixo para cada n.

9. Radixsort

```
void radixsort(int *v, unsigned int n, unsigned int maior) {
    int *v2 = malloc(n*sizeof(int));
    if(v2 == NULL) {
        return;
    }

    int *origem = v, *destino = v2;

    for(unsigned long long i = 1; i <= v[maior]; i *= 10) {
        unsigned int indice[10] = {0};
        for(unsigned int j = 0; j < n; ++j) {
            ++nComparacoes;
            ++indice[(origem[j]/i % 10)];
        }
        for(unsigned int j = 0; j < 9; ++j) {
            ++nComparacoes;
            indice[j+1] += indice[j];
            ++nMovimentacoes;
        }
        for(unsigned int j = n; j > 0; --j) {
            ++nComparacoes;
            unsigned int digito = (origem[j-1]/i % 10);
            destino[--indice[digito]] = origem[j-1];
            ++nMovimentacoes;
        }

        int *temp = origem;
        origem = destino;
        destino = temp;
    }

    if(origem != v) {
        for(unsigned int j = 0; j < n; ++j) {
            ++nComparacoes;
            v[j] = origem[j];
            ++nMovimentacoes;
        }
    }

    free(v2);
}
```

O *radixsort* é um método estável que utiliza a ideia de que, ao se comparar cada dígito de um número com o mesmo dígito de um outro, aquele com o maior algarismo é de maior

magnitude. Assim sendo, ele ordena aos poucos e depende não só do tamanho do vetor a ser ordenado, mas também da quantidade de dígitos do maior elemento do vetor.

Vetores ordenados:

Tamanhos	nComparações	nMovimentações	Tempo de execução (s)
100	727	427	0
1000	11.045	6.045	0
10000	150.063	80.063	0
100000	1.900.081	1.000.081	0,004

Vetores inversos:

Tamanhos	nComparações	nMovimentações	Tempo de execução (s)
100	727	427	0
1000	11.045	6.045	0
10000	150.063	80.063	0
100000	1.900.081	1.000.081	0,004

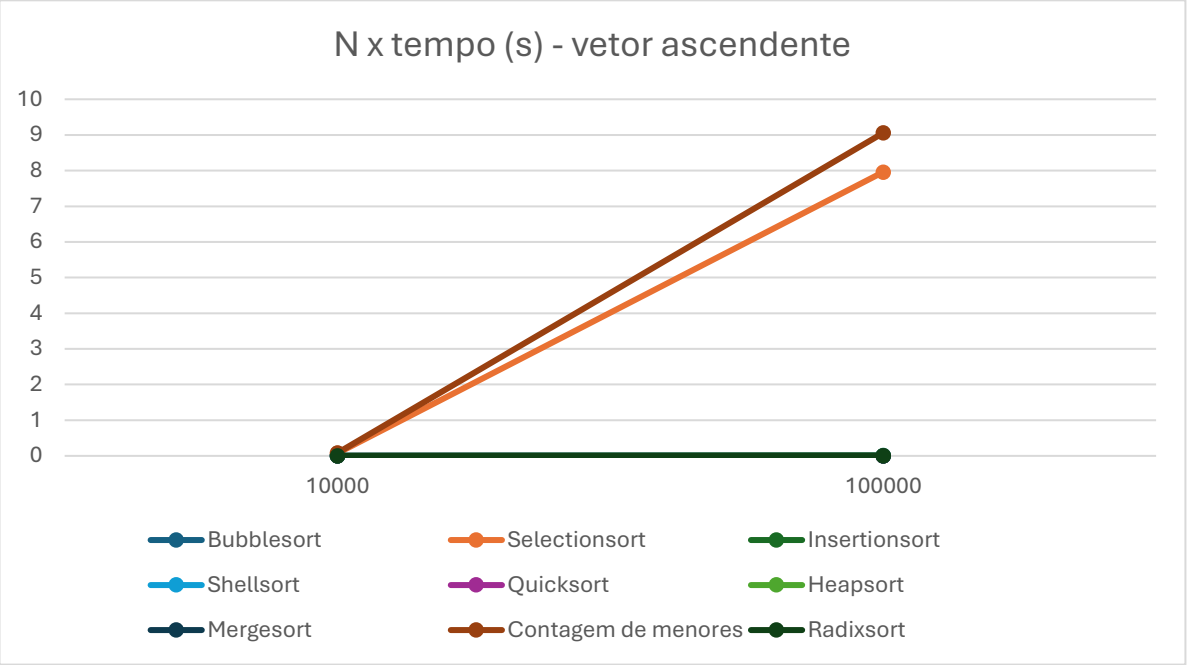
Vetores aleatórios:

Tamanhos	nComparações	nMovimentações	Tempo de execução (s)
100 (a)	727	427	0
100 (b)	727	427	0
100 (c)	727	427	0
100 (d)	727	427	0
100 (e)	727	427	0
1000 (a)	8.036	4.036	0
1000 (b)	8.036	4.036	0
1000 (c)	8.036	4.036	0
1000 (d)	8.036	4.036	0
1000 (e)	8.036	4.036	0
10000 (a)	110.045	60.045	0
10000 (b)	110.045	60.045	0
10000 (c)	110.045	60.045	0
10000 (d)	110.045	60.045	0
10000 (e)	110.045	60.045	0
100000 (a)	1.100.045	600.045	0,002
100000 (b)	1.100.045	600.045	0,002
100000 (c)	1.100.045	600.045	0,002
100000 (d)	1.100.045	600.045	0,002
100000 (e)	1.100.045	600.045	0,002

Note que o *radix* é O(n·d). Nos vetores aleatórios que geramos, acabamos nos deparando com o maior elemento sempre com a mesma quantidade de dígitos, então obtivémos os mesmos resultados para um certo *n*, mas poderíamos ter uma situação em que um dos vetores aleatórios teve um elemento com um dígito a mais do que os demais, por exemplo, o que o levaria a fazer mais comparações e modificações.

10. Comparações

a. Vetores ordenados

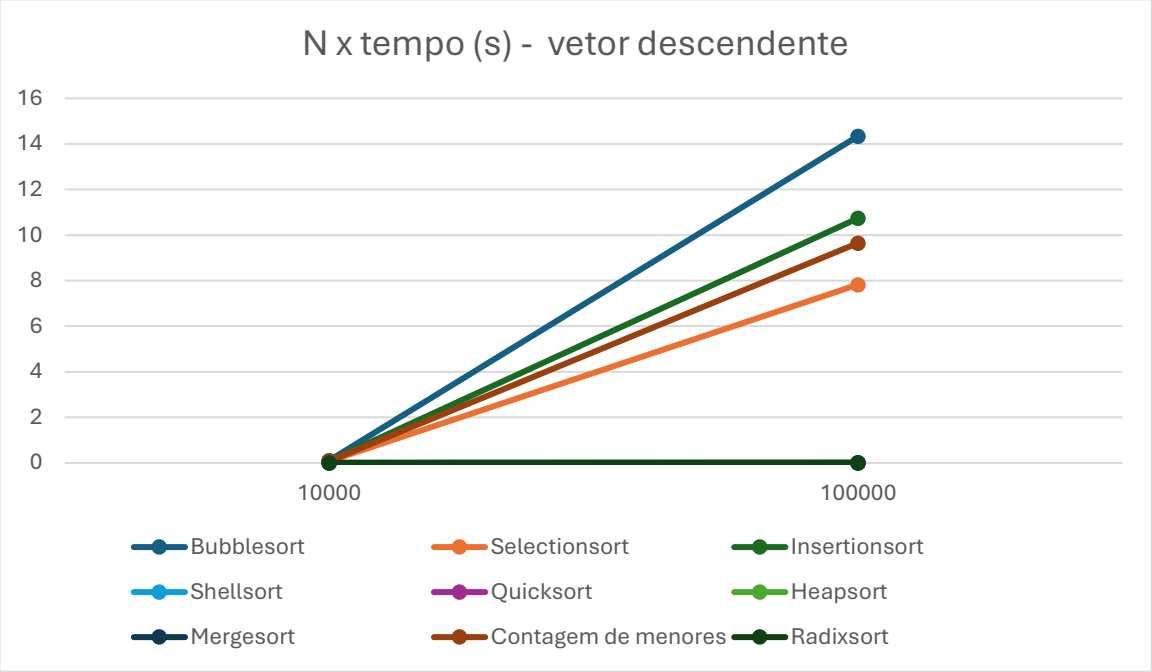


Perceba que o eixo vertical representa o tempo (em segundos) e o eixo horizontal o *n*. Como o tempo de execução para *n* = 100 e *n* = 1000 foi zero ou próximo de zero para todos, optamos por não colocar no gráfico para melhorar a visualização (o mesmo para a análise dos vetores inversamente e aleatoriamente ordenados).

Como é possível perceber, o tempo demorado é irrisório para quase todos os *sorts* nesse caso, à exceção do *Selectionsort* (7,96s) e da contagem de menores (9,057s). A explicação por isso é simples: o *Selectionsort* e a contagem de menores são ambos $O(n^2)$, independentemente do vetor original, enquanto os demais ou tem um big-oh menor ou o vetor ordenado representa seu melhor caso – como no *Bubblesort* e no *Insertionsort*, que são $\Omega(n)$ justamente quando o vetor já está ordenado.

Outros algoritmos, como o *Shellsort*, *Quicksort*, *Heapsort*, *Mergesort* e *Radixsort* não percorreram o vetor em $O(n)$, mas são tão eficientes que se aproximam muito mais do 0 do que do tempo levado pelos *Selectionsort* e pela contagem de menores.

b. Vetores inversamente ordenados

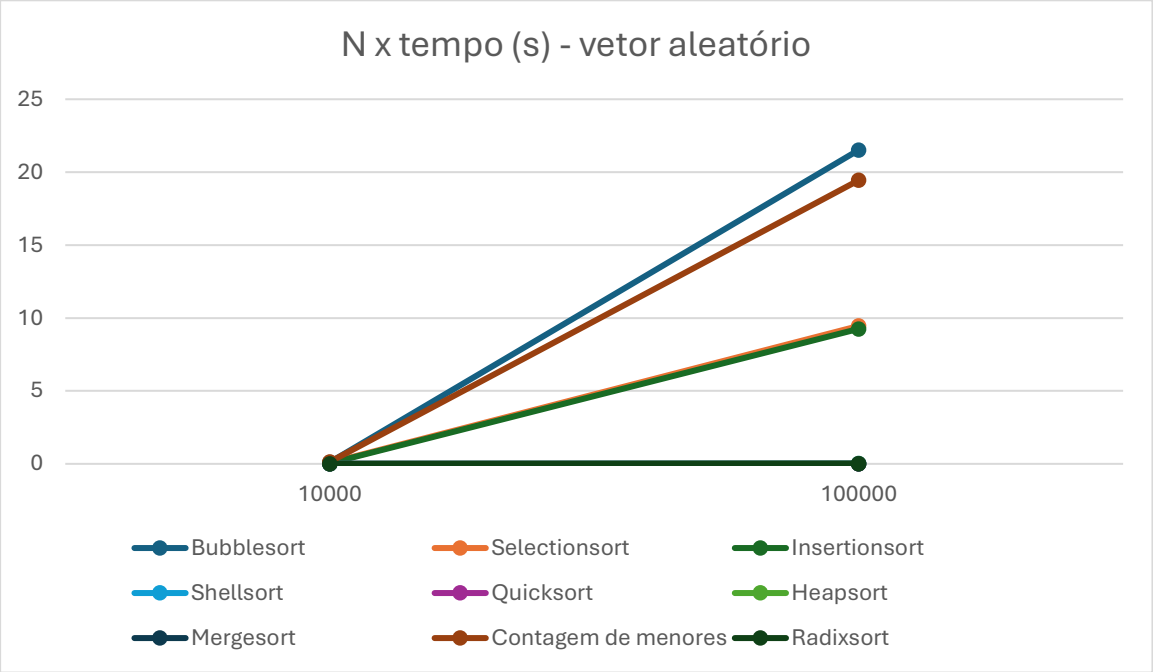


Aqui, já começamos a ver mais algoritmos levando mais tempo: permaneceram a contagem de menores e o *Selectionsort*, com tempos similares aos levados para os vetores ordenados, mas também apareceram o *Bubblesort* e o *Insertionsort*, dois algoritmos $O(n^2)$ quando o vetor está inversamente ordenado.

Vale comentar que, como o *Insertionsort* faz menos movimentações por deslocamento e não movimenta o mesmo elemento mais de uma vez, ele acaba sendo mais rápido que o *Bubblesort* na prática. Além disso, o *Selectionsort*, apesar de ser $\Theta(n^2)$, acaba se saindo na vantagem em relação aos demais em casos como esse, uma vez que ele faz pouquíssimas movimentações.

c. Vetores aleatoriamente ordenados

Para fazermos essa comparação, pegamos a média das 5 variações diferentes para cada *n*.



Aumentamos um pouco a média do *Selectionsort* porque ela deu igual à do *Insertion*, então não dava para vê-lo no gráfico.

Vemos que os piores algoritmos no caso aleatório foram o *Bubblesort*, contagem de menores, *Selectionsort* e *Insertionsort*. O *Selectionsort* e o método de contagem de menores são sempre $O(n^2)$, então o resultado era esperado. Já para o *Bubblesort* e *Insertionsort*, para o caso médio – ou aleatório –, podemos considerar que metade dos pares estão fora de ordem, obtendo $\frac{n(n-1)}{4}$, o que continua sendo $O(n^2)$! Novamente, o *Insertionsort* tem boa vantagem em comparação ao *bubble* porque faz bem menos movimentações.

Para visualizarmos melhor o que acontece, colocamos também uma tabela, já que o tempo de execução de alguns *sorts* se aproxima muito do zero, impedindo a visualização pelo gráfico.

Algoritmo	Tempo (s)
Bubblesort	21,5184
Contagem de menores	19,4432
Selectionsort	9,254
Insertionsort	9,254
Shellsort	0,0208
Heapsort	0,0188
Quicksort	0,0098
Mergesort	0,009
Radixsort	0,002

Dentre os mais rápidos, o *shell* e o *heap* foram os mais lentos. O *shell*, em grande parte, pela nossa escolha ingênua de h: escolhemos o h original, um dos piores conhecidos; ainda assim, seus resultados foram ótimos. Já o *heapsort* tem o maior custo constante dentre todos, uma vez que seu anel interno é mais custoso; por isso, ele acaba sendo um pouco mais lento que os demais.

Por outro lado, o *radixsort* se destaca por ser consideravelmente mais rápido que todos os outros. Isso ocorre porque o *radix* é $O(d \cdot n)$ e, da maneira que estruturamos a função de gerar, não chegamos a um número tão grande que seus dígitos se tornam um problema maior que a quantidade de elementos no vetor. O *radix* possui duas desvantagens, assim: a primeira que é necessário conhecer o índice do maior elemento (é necessário, então, para o caso aleatório, percorrer o vetor inteiro em busca desse elemento para, só então, poder fazer o *sort*, aumentando o custo real) e a segunda que, se trabalhássemos com números muito grandes, o *radix* seria menos recomendado que outros, como o *mergesort* iterativo que fizemos, o qual obteve resultados fantásticos.

d. Análise final

Assim sendo, o *radixsort* se mostra o mais rápido para elementos menores que a quantidade de elementos no vetor. Para situações mais gerais, o *mergesort* e o *quicksort* se mostraram os melhores, com menção honrosa ao *heapsort* e ao *shellsort* – o qual certamente seria ainda melhor com uma escolha mais adequada de h, como já observamos.

Em contrapartida, o *bubblesort* se demonstrou o pior algoritmo de todos, obtendo vantagem apenas quando o vetor já está ordenado; contudo, mesmo nesse seu melhor caso, ele empata com o *insertionsort*, o qual se dá melhor nas demais situações – apesar de também ter eficiência indesejável.

Além do *bubblesort*, o *selectionsort* e a contagem de menores também merecem menção honrosa por sua péssima eficiência. Sendo $\Theta(n^2)$, esses dois algoritmos têm pouquíssima aplicabilidade, mas acabam se saindo melhor que o *bubble* no caso aleatório, mais próximo da maioria das situações práticas.

11. Arquivo gerar.c

```
int main(void) {
   unsigned int ns[] = {100, 1000, 10000, 100000};
   unsigned int qtd = sizeof(ns)/sizeof(ns[0]);

   gerarordenado(ns, qtd);
   gerarinverso(ns, qtd);
   geraraleatorio(ns, qtd);
}

void gerarordenado(unsigned int *ns, unsigned int quant) {
   for(unsigned int i = 0; i < quant; ++i) {
      unsigned int n = ns[i];
      char filename[64];
      sprintf(filename, "input_%u-o.txt", n);
      FILE *f = fopen(filename, "w");
      if(!f) {
         perror("fopen");
         exit(1);
      }
      fprintf(f, "%u\n", n);
      for(unsigned int j = 1; j <= n; ++j) {
         unsigned int res = ((unsigned long long)j * j) / 100;
         fprintf(f, "%u\n", res);
      }
      fclose(f);
      printf("%s\n", filename);
   }
}

void gerarinverso(unsigned int *ns, unsigned int quant) {
   for(unsigned int i = 0; i < quant; ++i) {
      unsigned int n = ns[i];
      char filename[64];
      sprintf(filename, "input_%u-i.txt", n);

      FILE *f = fopen(filename, "w");
      if(!f) {
         perror("fopen");
```

```
        exit(1);
      }

      fprintf(f, "%u\n", n);
      for(unsigned int j = n; j >= 1; --j) {
        unsigned int res = (unsigned int)(((unsigned long long)j * j) / 100);
        fprintf(f, "%u\n", res);
      }
      fclose(f);
      printf("%s\n", filename);
    }
}

void geraraleatorio(unsigned int *ns, unsigned int quant) {
    for(unsigned int i = 0; i < 5; ++i) {
      for(unsigned int j = 0; j < quant; ++j) {
        unsigned int n = ns[j];
        char filename[64];
        sprintf(filename, "input_%u-%u-a.txt", n, i);

        FILE *f = fopen(filename, "w");
        if(!f) {
          perror("fopen");
          exit(1);
        }
        fprintf(f, "%u\n", n);
        unsigned int maxVal = 0;
        unsigned int maxIndex = 0;

        for(unsigned int k = 0; k < n; ++k) {
          unsigned int val = rand() % (n * 10);
          fprintf(f, "%u\n", val);

          if(k == 0 || val > maxVal) {
            maxVal = val;
            maxIndex = k;
          }
        }
        fclose(f);
        printf("%s INDICE DO MAIOR: %u\n", filename, maxIndex);
```

```
    }
  }
}
```

O arquivo gerar.c foi criado para facilitar a geração e utilização dos *inputs* requeridos. Ele gera, ao todo, 35 arquivos diferentes – 7 para cada tamanho de entrada, os quais podem ser utilizados como *input* durante a chamada do executável por meio do processo de tubulação.

Para gerar valores ordenados e inversamente ordenados, simplesmente criamos o arquivo com o nome correspondente e, depois, fazemos com que ele seja preenchido com os dados adequados, seguindo a equação previamente definida.

No caso aleatório, são gerados 5 arquivos diferentes contendo uma entrada gerada aleatoriamente pela função *rand()* da biblioteca *stdlib*. Foi necessário criar os 5 arquivos numa mesma execução para permitir uma diversificação dos valores randômicos, visto que a função *rand()* tem uma *seed* padrão; ou seja, se reiniciássemos a execução, geraríamos sequências idênticas.

Ainda no aleatório, devolvemos ao usuário o índice que contém o maior elemento para uso no *radixsort*. Numa situação real, teríamos que percorrer o vetor para achar o índice do maior para o método funcionar. Nas demais funções de gerar, isso não é feito, pois o maior elmento sempre estará na primeira ou na última posição.

Os arquivos resultantes são dispostos de tal forma que podem ser perfeitamente usados como *input*, porque a primeira linha de cada um possui o tamanho do vetor que será inserido e as demais linhas são a sequência utilizada.